

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 文件操作

实验学生班级 信息 171

实验学生姓名 骆舒 208170826

同组学生姓名

实 验 时 间 2020.6.20

实 验 地 点

实验成绩评定

指导教师签字

年 月 日

一、实验目的和要求

二、实验主要仪器和设备

三、实验原理

四、实验内容过程及结果

五、实验中遇到问题及解决方法

六、实验心得：

一、实验目的

掌握Linux系统中C编写应用程序的方法。学习Linux的文件系统的程序接口，掌握与文件和目录有关的操作。

二、实验主要仪器和设备

硬件：PC机

软件：VM虚拟机、Red Hat.

三、实验内容

应用程序编辑、编译、链接、运行、调试的过程，针对目录和文件的编程操作。

1. hello world

vim helloworld.c

// 编译.c文件

gcc helloworld.c -o helloworld //生成可执行文件

./helloworld

// 执行文件

实验结果：

```
[root@cvtech ~]# vim helloworld.c
```

```
[root@cvtech ~]# gcc helloworld.c -o helloworld
```

```
[root@cvtech ~]# ./helloworld
```

```
Hello,Word!
```

2. who.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <utmp.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
void show(long timeval)
```

```
{
```

```

char *cp = ctime (&timeval); // 转化时间
printf ("%12.16s", cp);
}
int show_info (struct utmp *utbufp)
{
    if (utbufp->ut_type != USER_PROCESS) // 当不为用户进程时,不输出
    {
        return 0;
    }
    printf ("% - 8.8s", utbufp->ut_name); // 格式化输入信息
    printf ("% - 8.8s", utbufp->ut_line);
    showTime (utbufp->ut_time);
#ifdef SHOWHOST
    if (utbufp->ut_host[0] != '\0') // 定义了SHOWHOST且有值时
        printf ("%s", utbufp->ut_host); // 输出信息
#endif
    printf ("\n");
    return 0;
}
int main ()
{
    struct utmp current_record; // 读取信息格式
    int utmpfd;
    int reclen = sizeof (current_record);
    if ((utmpfd = open (UTMP_FILE, O_RDONLY)) == -1)
    {
        perror (UTMP_FILE); exit (1);
    } // 报出系统错误
    while (read (utmpfd, &current_record, reclen) == reclen)
    {
        show_info (&current_record); // 读取文件信息,写出信息
    }
    close (utmpfd);
    return 0;
}

```

结果:

```

[root@cvtech ~]# vim who.c
[root@cvtech ~]# gcc who.c -o who
[root@cvtech ~]# ./who
root      :0      Sun Jun 21 23:23
root      pts/0    Sun Jun 21 23:23 (:0.0)

```

3. cp.c

```

#define COPYMODE 0644 // 文件权限
#define BUF 4096 // 读取字符串长度

```

```

int main (int argc, char *argv[]) // 命令行模式
{
    if (argc != 3) // 当命令行参数不为3时, 输出错误信息.
    {
        printf("usage: %s source destination\n", argv[0]);
        exit(-1); // 异常退出
    }
    int in_fd = -1;
    int out_fd = -1;
    if ((in_fd = open(argv[1], O_RDONLY)) == -1) // 打开文件失败
    {
        perror("file open"); exit(-1);
    }
    if ((out_fd = creat(argv[2], COPYMODE)) == -1) // 创建文件失败
    {
        perror("file copy"); exit(-1);
    }
    char n_chars[BUF];
    int len = 0;
    while ((len = read(in_fd, n_chars, sizeof(n_chars))) > 0) // 复制文件
    {
        if (write(out_fd, n_chars, len) != len) // 当读写长度不等, 则为失败
        {
            printf("copy error\n", argv[2]); exit(-1);
        }
    }
    if (len == -1)
    {
        printf("read error\n", argv[1]); exit(-1);
    }
    if (close(in_fd) == -1) // 关闭文件失败
    {
        printf("close error\n", argv[1]); exit(-1);
    }
    if (close(out_fd) == -1)
    {
        printf("close error\n", argv[2]); exit(-1);
    }
    return 0;
}

```

结果:

```

[root@cvtech ~]# vim cp.c
[root@cvtech ~]# gcc cp.c -o cp
[root@cvtech ~]# ls
alarm2          hello_multi    myls
alarm2.c        hello_multi.c  myls.c
anaconda-ks.cfg helloworld     mytree
cli             helloworld.c  mytree.c
cli.c           install.log    new-prcs
cp              install.log.syslog new-prcs.c
cp.c            leak           parent-child

```

```

[root@cvtech ~]# ./cp helloworld.c copy1.c
[root@cvtech ~]# vim copy1.c
[root@cvtech ~]# ls
alarm2          hello.c          Music
alarm2.c        hello_multi      myls
anaconda-ks.cfg hello_multi.c    myls.c
cli             helloworld       mytree
cli.c           helloworld.c     mytree.c
copy1.c         install.log       new-prcs
cp              install.log.syslog new-prcs.c
cp.c            leak             parent-child

```

```

#include <stdio.h>
#include <stdlib.h>
int main()
{
    printf("Hello, Word!\n");
    return 0;
}

```

4. tree.c

```

#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>
void scan_dir(char *dir, int depth)
{
    DIR *dp;
    struct dirent *entry;
    struct stat statbuf;
    if ((dp = opendir(dir)) == NULL) { // 如果打开目录失败
        puts("Cannot open directory"); // 输出错误信息
        return;
    }
    chdir(dir);
    while ((entry = readdir(dp)) != NULL) { // 如果读目录成功
        lstat(entry->d_name, &statbuf);
        if (S_IFDIR & statbuf.st_mode) {
            if (strcmp(".", entry->d_name) == 0 || strcmp("..", entry->d_name) == 0)
                continue;
            printf("%s%s\n", depth, "", entry->d_name);
            scan_dir(entry->d_name, depth + 1); // 递归
        }
        else
            printf("%s%s\n", depth, "", entry->d_name);
    }
}

```

```

    chdir ("..");
    closedir (dp); //完成后关闭文件
int main (int argc, char *argv[]) //命令行参数
{
    puts ("Scan the directory:");
    scan_dir (argv[1], 0); //输出
    puts ("Scanning ended.");
    return 0;
}

```

结果:

```

[root@cvtech ~]# vim tree.c
[root@cvtech ~]# gcc tree.c -o tree
[root@cvtech ~]# ./tree Desktop
Scan the directory:
gnome-terminal.desktop
Scanning ended.

```

5. myls.c

```

void do_ls (char dirname[])
{
    DIR * dir_ptr;
    struct dirent * direntp;
    if ((dir_ptr = opendir (dirname)) == NULL) //文件夹内不为空
        fprintf (stderr, "ls: cannot open %s\n", dirname);
    else {
        while ((direntp = readdir (dir_ptr)) != NULL)
            printf ("%s\n", direntp->d_name); //输出用户名
        closedir (dir_ptr);
    }
}

int main (int ac, char * av[])
{
    if (ac == 1)
        do_ls ("."); //当仅执行 ls 时,只展开当前文件夹中内容
    else
        while (--ac) { //当后面跟文件夹时,展开文件夹中的内容
            printf ("%s:\n", *++av);
            do_ls (*av);
        }
}

```

结果:

```

[root@cvtech ~]# vim myls.c
[root@cvtech ~]# gcc myls.c -o myls
[root@cvtech ~]# myls

```

```
bash: myls: command not found
[root@cvtech ~]# ./mysls
.redhat
.gconf
write2cli
Music
copy1.c
tree
write2cli.c
anaconda-ks.cfg
.tcshrc
.local
.pulse-cookie
parent-child
mytree
execl
segfault.c
segfault
.gconfd
.gtk-bookmarks
cp
```

四、实验总结

1. 熟悉了 vim, gcc 等命令的使用;
2. 基本掌握了 Linux 中文件与目录的操作;
3. 熟悉了 Linux 编程环境, 为以后的实验打下基础。

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 进程操作

实验学生班级 信息171

实验学生姓名 骆舒 208170826

同组学生姓名 _____

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一、实验目的和要求

二、实验主要仪器和设备

三、实验原理

四、实验内容过程及结果

五、实验中遇到问题及解决方法

六、实验心得:

一、实验目的

掌握进程概念,熟悉进程状态及其转换,学习进程控制相关的系统调用与函数。

二、实验主要仪器和设备

硬件:

PC机

软件: VM 虚拟机

三、实验内容

1. parent-child.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
int main(void)
```

```
{
    pid_t pid;           // 定义该结构保存子进程的PID
    printf("-----\n");
    pid = fork();         // 创建子进程
    if (pid == -1)
    {
        perror("fork error"); exit(1);
    }
    else if (pid == 0)    // 输出子进程的PID
    {
        printf("I am child process, pid=%u, ppid=%u", getpid(), getppid());
    }
    else                 // 输出父进程的PID
    {
        printf("I am child process, pid=%u, ppid=%u\n", getpid(), getppid());
        sleep(1);
    }
    return 0;
}
```

```
[root@cvtech ~]# vim parent-child.c
[root@cvtech ~]# gcc parent-child.c -o parent-child
[root@cvtech ~]# ./parent-child
```

```
-----
I am child process,pid=4445,ppid=4444
I am parent process,pid=4444,ppid4021
```

2. execl.c

```
#include <sys/stat.h>
#include <sys/types.h>

void scan_dir (char *dir, int depth)
{
    DIR *dp;
    struct dirent *entry;
    struct stat statbuf;
    if ((dp = opendir (dir)) == NULL) { // 文件夹内容不为空
        puts ("Cannot open directory"); // 输出错误信息
        return; }
    chdir (dir);
    while ((entry = readdir (dp)) != NULL) { // 如果读目录成功
        lstat (entry->d_name, &statbuf);
        if (S_IFDIR & statbuf.st_mode) {
            if (strcmp (".", entry->d_name) == 0 || strcmp ("..", entry->d_name) == 0)
                continue;
            printf ("%s%s\n", depth, " ", entry->d_name);
            scan_dir (entry->d_name, depth+4); } // 递归
        else
            printf ("%s%s\n", depth, " ", entry->d_name); }
    chdir ("..");
    closedir (dp); // 完成后关闭文件
}

int main (int argc, char *argv[]) // 命令行参数
{
    puts ("Scan the directory:");
    scan_dir (argv[1], 0); // 输出
    puts ("Scanning ended.");
    return 0;
}
```

结果:

```
[root@cvtech ~]# vim execl.c
[root@cvtech ~]# gcc execl.c -o execl
[root@cvtech ~]# ./execl Desktop
Scan the directory:
gnome-terminal.desktop
Scanning ended.
```

3. new-prcs.c

```
int main (void)
{
    pid_t pid;
    if ((pid = fork()) < 0) // 创建进程失败
    {
        perror("fork error"); exit(1); // 输出错误信息
    }
    if (pid == 0) // 子进程中
    {
        execl("/bin/date", "date", (char *) 0);
        perror("execl() failure!\n\n"); // 输出错误信息
    } else {
        printf("PARENT PROCESS.\n"); // 父进程
    }
    return (0);
}
```

结果:

```
[root@cvtech ~]# vim new-prcs.c
[root@cvtech ~]# gcc new-prcs.c -o new-prcs
[root@cvtech ~]# ./new-prcs
There is only one process now.
This message is printed by the child process.
Sun Jun 21 23:58:00 CST 2020
PARENT PROCESS.
===The End===
```

3. segfault.c

```
int main (int argc, char **argv)
{
    int a; a = *(int *)0; // 读地址
}
```

结果:

```
[root@cvtech ~]# vim segfault.c
[root@cvtech ~]# gcc segfault.c -o segfault
[root@cvtech ~]# ./segfault
Segmentation fault
```

4. alarm2.c

```
void sig_handler (int signo)
{
    alarm(3);
}

int main (void)
{
    if (signal(SIGALRM, sig_handler) == SIG_ERR)
        printf("\n can't catch SIGALRM\n"); // 输出错误信息
    alarm(3);
    long i=0;
```

```

while (1)           // 循环
{
    sleep (1);
    i++;
}
return 0;
}

```

结果.

```

[root@cvtech ~]# vim alarm2.c
[root@cvtech ~]# gcc alarm2.c -o alarm2
[root@cvtech ~]# ./alarm2
I am doing hard word. i=0 now.
I am doing hard word. i=1 now.
I am doing hard word. i=2 now.

```

B-e-e-p! 3 sencond passed.

```

I am doing hard word. i=3 now.
I am doing hard word. i=4 now.
I am doing hard word. i=5 now.

```

B-e-e-p! 3 sencond passed.

3.

四.总结

1. 基本掌握了进程的概念,熟悉进程状态及其转换;
2. 熟悉了 Linux 进程控制相关的系调用与函数;
3. 熟悉了进程间的相互联系和关系。

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 多线程与内存管理

实验学生班级 信息171

实验学生姓名 骆舒 208170826

同组学生姓名

实 验 时 间 2020.6.20

实 验 地 点

实验成绩评定

指导教师签字

年 月 日

一、实验目的和要求

二、实验主要仪器和设备

三、实验原理

四、实验内容过程及结果

五、实验中遇到问题及解决方法

六、实验心得：

一、实验目的

掌握线程概念,熟悉进程状态及其转换,学习线程控制相关的系统调用与函数。

二、实验主要仪器和设备

硬件: PC 机

软件: VM虚拟机

三、实验内容

1. hello_multi

```
void *print_msg(void *m)
{
    char *cp = (char *)m;
    int i;
    for(i=0; i<NUM; i++)    // 五次循环
    {
        printf("%s", cp);
        fflush(stdout);
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;        // 定义两个线程结构
    void *print_msg(void *);
    pthread_create(&t1, NULL, print_msg, (void *)"hello"); // 创建线程1
    pthread_create(&t2, NULL, print_msg, (void *)"world\n"); // 创建线程2
    pthread_join(t1, NULL); // 等待一个线程的结束
    pthread_join(t2, NULL);
}
```

结果:

```
[root@cvtech ~]# vim hello_multi.c
[root@cvtech ~]# gcc hello_multi.c -o hello_multi -lpthread
[root@cvtech ~]# ./hello_multi
helloworld
helloworld
world
hellohelloworld
world
```

2. twordcount1.c.

```
void *count_words(void *f)
{
    char *filename = (char *)f;
    FILE *fp;
    int c, prevc = '\0';
    if ((fp = fopen(filename, "r")) != NULL)
    {
        while ((c = getc(fp)) != EOF)
        {
            if (!isalnum(c) && isalnum(prevc))
                total_words++; // 计数
            prevc = c;
        }
        fclose(fp); // 关闭文件
    } else
        perror(filename);
    return NULL;
}

int total_words;
int main(int ac, char *av[])
{
    pthread_t t1, t2; // 定义两个线程
    void *count_words(void *);
    if (ac != 3) // 当命令行参数不为3时, 输出错误信息
    {
        printf("usage: %s file1 file2\n", av[0]); exit(1);
    }
    total_words = 0;
    pthread_create(&t1, NULL, count_words, (void *) av[1]); // 创建线程 t1
    pthread_create(&t2, NULL, count_words, (void *) av[2]); // 创建线程 t2
    pthread_join(t1, NULL); // 等待子线程 t1 结束
    pthread_join(t2, NULL); // 等待子线程 t2 结束
    printf("%d: total words\n", total_words);
}
```

结果:

```
[root@cvtech ~]# vim twordcount1.c
[root@cvtech ~]# gcc twordcount1.c -o twordcount1 -lpthread
[root@cvtech ~]# ./twordcount1 who.c myls.c
210:total words
```

3. twordcount2.c

```
int total_words;
pthread_mutex_t counter_lock = PTHREAD_MUTEX_INITIALIZER; //初始化互斥锁
int main(int ac, char *av[])
{
    pthread_t t1, t2; //定义两个线程结构 t1, t2
    void *count_words(void *);
    if (ac != 3) //当命令行参数不为3时, 输出错误信息
    {
        printf("usage: %s file1 file2\n", av[0]); exit(1);
    }
    total_words = 0;
    pthread_create(&t1, NULL, count_words, (void *) av[1]); //创建线程 t1
    pthread_create(&t2, NULL, count_words, (void *) av[2]); //创建线程 t2
    pthread_join(t1, NULL); //等待子线程 t1 结束
    pthread_join(t2, NULL); //等待子线程 t2 结束
    printf("%5d: total words\n", total_words);
}

void *count_words(void *f)
{
    char *filename = (char *)f; //文件名
    FILE *fp;
    int c, prevc = '\0';
    if ((fp = fopen(filename, "r")) != NULL) //如果打开文件成功
    {
        while ((c = getc(fp)) != EOF)
        {
            if (!isalnum(c) && !isalnum(prevc))
            {
                pthread_mutex_lock(&counter_lock); //锁住互斥量
                total_words++;
                pthread_mutex_unlock(&counter_lock);
            }
            prevc = c;
        }
        fclose(fp); //完成后关闭文件
    }
    else
        perror(filename);
    return NULL;
}
```

结果:

```
[root@cvtech ~]# vim twordcount2.c
[root@cvtech ~]# gcc twordcount2.c -o twordcount2 -lpthread
[root@cvtech ~]# ./twordcount2 who.c myls.c
210:total words_
```

4. leak.c

```
#include <stdio.h>
#include <stdlib.h>
void f(void)
{
    void *s;
    s = malloc(50);    //分配内存资源
    return;
}
int main(void)
{
    while(1) f();    //循环
    return 0;
}
```

结果:

```
[root@cvtech ~]# vim leak.c
[root@cvtech ~]# gcc leak.c -o leak -lpthread
[root@cvtech ~]# ./leak
Killed
```

5. lockdemo

```
int main(int argc, char *argv[])
{
    struct flock fl = { F_WRLCK, SEEK_SET, 0, 0, 0 }; //定义结构体
    int fd;
    fl.l_pid = getpid();    //获取进程PID
    if (argc > 1)
        fl.l_type = F_RDLCK;
    if ((fd = open("lockdemo.c", O_RDWR)) == -1) //如果打开文件失败
    {
        perror("open"); exit(1);
    }
    printf("Press <RETURN> to try to get lock:");
    getchar();
    if (fcntl(fd, F_SETLKW, fl) == -1)
    {
        perror("fcntl"); exit(1);
    }
}
```

```

printf ("got lock\n");
getchar();
fl.l_type = F_UNLCK;
if (fcntl (fd, F_SETLK, fl) == -1)
{ perror ("fcntl"); exit(1); } //输出错误信息
printf ("Unlocked.\n");
close (fd); //完成后关闭
return 0;
}

```

结果:

```

[root@cvtech ~]# vim lockdemo.c
[root@cvtech ~]# gcc lockdemo.c -o lockdemo -lpthread
[root@cvtech ~]# ./lockdemo
Press <RETURN> to try to get lock:
Trying to get lock...got lock
Press <RETURN> to release lock:
Unlocked.

```

6. semaphores.c

```

#define TEN_MILLION 1000000L
#define BUFSIZE 1024 //定义缓存大小
#define MAXPIDS 10

void worker (int i, void *args)
{
    char buffer [BUFSIZE];
    char *c;
    sem_t *semlockp;
    struct timespec sleeptime; //定义结构体
    semlockp = (sem_t *) args;
    sleeptime.tv_sec = 0;
    sleeptime.tv_nsec = TEN_MILLION;
    snprintf (buffer, BUFSIZE, "This is process %ld\n", (long) getpid());
    c = buffer;
    setbuf (stderr, NULL); //为stderr指定无缓冲
    while (sem_wait (semlockp) == -1) //入口部分
    {
        if (errno != EINTR) {
            fprintf (stderr, "Thread failed to lock semaphore\n"); //输出错误信息
            return;
        }
    }
}

```

```

while (*c != '\0') {
    fputc(*c, stderr); //无缓冲,同步输出
    c++;
    nanosleep(& sleeptime, NULL);
}
if (sem_post(semlock) == -1) //退出部分
    fprintf(stderr, "Thread failed to unlock semaphore\n");
return;
}

int main(int argc, char *argv[]) { //命令行参数
    int error;
    int i;
    int id, n;
    sem_t *semlock;
    pid_t pids[MAXPIDS];
    if (argc != 2) { //检查命令行参数的有效数目
        fprintf(stderr, "usage: %s numprocesses\n", argv[0]);
        return 1;
    }
    n = atoi(argv[1]);
    if (n > MAXPIDS)
        return 1;
    #ifdef USE_SHARED_MEMORY //如无法创建共享内存段
        if ((id = shmget(IPC_PRIVATE, sizeof(sem_t), (S_IRUSR | S_IWUSR))) == -1)
        {
            perror("Failed to create shared memory segment");
            return 1; //输出错误信息
        }
    #else //尝试对 sem_t 变量使用普通进程专用内存
        semlock = (sem_t *) malloc(sizeof(sem_t));
    #endif
    if (sem_init(semlock, 1, 1) == -1) { //如无法附加内存段
        perror("Failed to initialize semaphore"); //输出错误信息
    } else {
        for (i = 0; i < n; i++) {
            if ((pids[i] = fork()) < 0) { //创建进程失败
                fprintf(stderr, "Failed to create process: %s\n", strerror(errno));
                return 1; //输出错误信息
            }
        }
    }
}

```

```

    if (pids[i] == 0) { // 子进程开始
        worker(i, semlock);
        exit(0);
    }
}
for (i=0; i<n; i++) // 所有的父进程
    wait(0);
fprintf(stderr, "workers all done\n");
#ifdef USE_SHARED_MEMORY
    if (shmdt((void *) semlock == -1) { // 共享内存
        perror("Failed to destroy shared memory segment");
        return 1;
    }
#else
    free(semlock);
#endif
return 0;
}

```

结果:

```

[root@cvtech ~]# vim semaphores.c
[root@cvtech ~]# gcc semaphores.c -o semaphores -lpthread
[root@cvtech ~]# ./semaphores
Usage: ./semaphores numprocesses
[root@cvtech ~]# ./semaphores 5
TTTTThhhhhiiiiisssss      iiiisssss      ppppprrrrrrroooooocccccceee
44444555558887829301

```

workers all done

总结:

1. 基本掌握了多线程与内存管理的概念, 熟悉多线程状态, 以及共享内存的控制使用。
2. 熟悉了 Linux 中多线程与内存管理的操作, 进程的编译需要加上参数 `-lpthread`, 否则提示找不到函数的错误。

南京工程学院

信息与通信工程学院

实 验 报 告

课 程 名 称 嵌入式系统编程

实验项目名称 项目管理与综合实验

实验学生班级 信息171

实验学生姓名 骆舒 208170826

同组学生姓名 _____

实 验 时 间 2020.6.20

实 验 地 点 _____

实验成绩评定 _____

指导教师签字 _____

年 月 日

一、实验目的和要求

二、实验主要仪器和设备

三、实验原理

四、实验内容过程及结果

五、实验中遇到问题及解决方法

六、实验心得：

一、实验目的

掌握套接字概念,熟悉网络编程方法,学习网络编程相关的系统调用与函数。

二、实验主要仪器和设备

硬件: PC机

软件: VM虚拟机

三、实验内容

1. gethostname.c

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>

int main (int argc, char *argv[])
{
    char *hostname = "www.163.com";
    char ip[100];
    struct hostent *he;
    struct in_addr **addr_list;
    int i;
    if ((he = gethostbyname(hostname)) == NULL)
    {
        perror("gethostbyname"); // 输出错误信息
        return 1;
    }
    addr_list = (struct in_addr **) he->h_addr_list; // 将 h_addr_list 转换为 in_addr
    for (i=0; addr_list[i] != NULL; i++)
    {
        strcpy(ip, inet_ntoa(*addr_list[i])); // 复制字符串
        printf("%s resolved to: %s\n", hostname, ip);
    }
    return 0;
}
```

结果:

```
[root@cvtech ~]# vim gethostbyname.c
[root@cvtech ~]# gcc gethostbyname.c -o gethostbyname -lpthread
[root@cvtech ~]# ./gethostbyname
www.163.com resolved to : 36.155.87.42
www.163.com resolved to : 36.156.21.10
www.163.com resolved to : 36.155.87.39
www.163.com resolved to : 36.156.21.13
www.163.com resolved to : 36.155.87.45
www.163.com resolved to : 36.156.21.16
www.163.com resolved to : 36.155.87.43
www.163.com resolved to : 36.156.21.5
www.163.com resolved to : 36.155.87.37
www.163.com resolved to : 36.156.21.7
```

2. multi-server.c

```
void *connection_handler(void *socket_desc)
{
    int sock = *(int *) socket_desc; //获取套接字描述符
    int read_size;
    char *message, client_message[2000];
    message = "Greeting! I am your connection handler\n"; //发送信息到客户端
    write(sock, message, strlen(message));
    while ((read_size = recv(sock, client_message, 2000, 0)) > 0) //从客户端
    {
        write(sock, client_message, strlen(client_message)); //接收信息
    }
    if (read_size == 0) //客户端连接失败
    {
        puts("client disconnected"); //输出错误信息
        fflush(stdout);
    }
    else if (read_size == -1)
    {
        perror("recv failed");
    }
    free(socket_desc); //释放套接字指针
    return 0;
}

int main(int argc, char *argv[])
{
    int socket_desc, new_socket, c, *new_sock;
    struct sockaddr_in server, client;
    char *message;
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); //创建套接字
    if (socket_desc == -1) //创建套接字失败
    {
        printf("could not create socket"); //输出错误信息
    }
}
```

```

server.sin_family = AF_INET; // 准备 sockaddr_in 结构
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(8888);
if (bind(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0)
{
    puts("bind failed"); // 如果绑定套接字失败, 输出错误信息
    return 1;
}

listen(socket_desc, 3);
puts("waiting for incoming connections..."); // 接受和传入连接
c = sizeof(struct sockaddr_in);
while ((new_socket = accept(socket_desc, (struct sockaddr *)&client,
(socket_t *)&c)))
{
    puts("Connection accepted");
    message = "Hello Client. I have received your connection. And now I
will assign a handler for you\n"; // 回复客户端
write(new_socket, message, strlen(message));
pthread_t sniffer_thread;
new_sock = malloc(1);
*new_sock = new_socket;
if (pthread_create(&sniffer_thread, NULL, connection_handler, (void *)
new_sock) < 0) // 如果创建线程失败
{
    perror("could not create thread"); // 输出错误信息
    return 1;
}
puts("Handler assigned");
}
if (new_socket < 0)
{
    perror("accept failed");
    return 1;
}
return 0;
}

```

结果:

```

[root@cvtech ~]# vim multi-server-2.c
[root@cvtech ~]# gcc multi-server-2.c -o multi-server-2 -lpthread
[root@cvtech ~]# ./multi-server-2
bind done
Waiting for incoming connections...

```

3. serv.c

```
void *connection_handler(void *socket_desc)
{
    int sock = *(int *) socket_desc; // 获取套接字描述符
    int read_size;
    char *message, client_message[2000];
    write(sock, message, strlen(message));
    write(sock, message, strlen(message));
    while ((read_size = recv(sock, client_message, 2000, 0)) > 0)
    {
        int i;
        // 从客户端接收信息
        for (i = 0; i < strlen(client_message); i++) {
            printf("%02x", client_message[i]); // 发送信息回客户端
            fflush(stdout);
        }
    }
    if (read_size == 0) // 客户端连接失败
    {
        puts("client disconnected"); // 输出错误信息
        fflush(stdout);
    }
    else if (read_size == -1)
    {
        perror("recv failed");
    }
    free(socket_desc); // 释放套接字指针
    return 0;
}
```

```
int main(int argc, char *argv[])
{
    int socket_desc, new_socket, c, *new_sock;
    struct sockaddr_in server, client;
    char *message;
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); // 创建套接字
    if (socket_desc == -1) // 创建失败
    {
        printf("could not create socket"); // 输出错误信息
    }
    server.sin_family = AF_INET; // 准备 sockaddr_in 结构
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8888);
```

```

if (bind (socket_desc, (struct sockaddr *)&server, sizeof (server)) < 0)
{
    puts ("bind failed"); //如果绑定套接字失败, 输出错误信息.
    return 1;
}
puts ("bind done");
listen (socket_desc, 3);
puts ("waiting for incoming connections ..."); //接受和传入连接
c = sizeof (struct sockaddr_in);
while ( (new_socket = accept (socket_desc, (struct sockaddr *)&client,
(socklen_t *)&c))
{
    puts ("connection accepted");
    write (new_socket, message, strlen (message));
    pthread_t sniffer_thread;
    new_sock = malloc (1);
    *new_sock = new_socket;
    if (pthread_create (&sniffer_thread, NULL, connection_handler, (void *)
new_sock) < 0) //如果创建线程失败
    {
        perror ("could not create thread"); //输出错误信息
        return 1;
    }
    puts ("Handler assigned");
}
if (new_socket < 0)
{
    perror ("accept failed");
    return 1;
}
return 0;
}

```

4. cli.c

```

int main (int argc, char *argv[])
{
    int socket_desc, fd;
    struct sockaddr_in server;
    char message [1024];
    socket_desc = socket (AF_INET, SOCK_STREAM, 0); //创建套接字
}

```

```

if (socket_desc == -1) //如果创建套接字失败
{
    printf ("Could not create socket"); } //输出错误信息
server.sin_addr.s_addr = inet_addr(argv[1]); //准备 sockaddr_in 结构
server.sin_family = AF_INET;
server.sin_port = htons(8888);
if (connect(socket_desc, (struct sockaddr *)&server, sizeof(server)) < 0)
{
    puts ("connect error"); //连接到远程服务器, 如果连接失败
    return 1; //返回错误信息
}
puts ("Connected \n");
if ((fd = open("/dev/input/mice", O_RDONLY)) == -1) //从 mice 读取数据
{
    perror ("cannot open /dev/input/mice"); //从套接字将其发送到客户端
    return 1;
}
while (read(fd, message, 1024) > 0)
{
    if (send(socket_desc, message, strlen(message), 0) < 0)
    {
        puts ("Send failed"); //如果发送失败, 输出错误信息
        return 1;
    }
}
return 0;
}

```

结果:

```

root@cvtech:~
File Edit View Terminal Tabs Help
[root@cvtech ~]# vim serv.c
[root@cvtech ~]# gcc serv.c -o serv -lpthread
[root@cvtech ~]# ./serv 8888
bind done
Waiting for incoming connections...
Connection accepted
Handler assigned
28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF
1 FFFFFFFF 28 01 FFFFFFFF 28 03 FFFFFFFD 28
FFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FF
28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF
1 FFFFFFFF 28 02 FFFFFFFE 28 01 FFFFFFFF 28
FFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF 28 02 FF
28 01 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF
1 FFFFFFFF 28 01 FFFFFFFF 28 01 FFFFFFFF 28
root@cvtech:~
File Edit View Terminal Tabs Help
cli -lpthread
0.1 8888
[root@cvtech ~]# vim cli.c
[root@cvtech ~]# gcc cli.c -o
[root@cvtech ~]# ./cli 127.0.0.1
Connected

```

四. 总结:

1. 基本掌握了套接字概念, 熟悉网络编程方法;
2. 熟悉了 Linux 中网络编程相关的系统调用与函数;
3. 了解了客户端与服务器间的工作流程: 客户端程序首先通过服务器获得服务器的 IP 地址, 然后创建一个 socket, 调用 `connect()` 函数与服务器建立连接, 连接成功后接收从服务器发送来的数据, 最后关闭 socket.